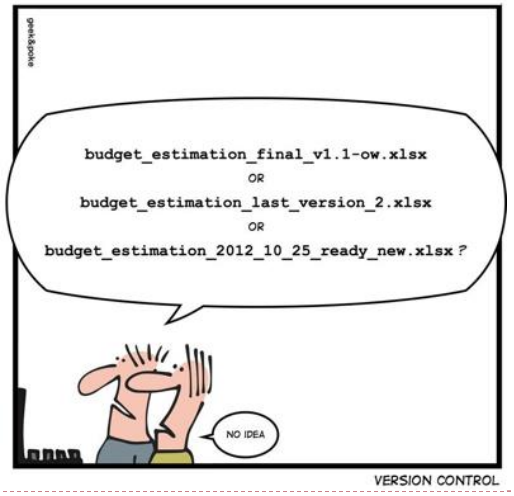
12. Software Version Control

"Software Development Methodologies"

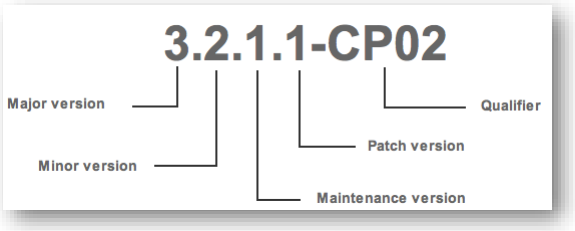
Ricardo Filipe, ISEC (ricardo.filipe@isec.pt)

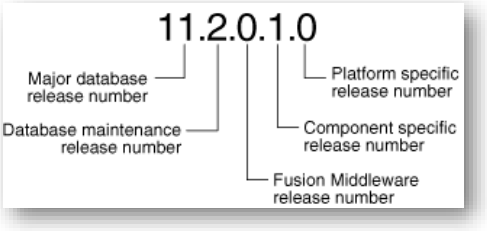
391

SIMPLY EXPLAINED



392





393

393

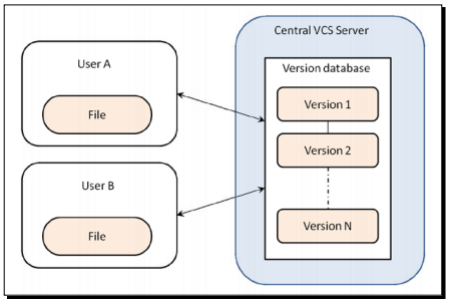
Configuration Management

- Configuration Management (CM) is the discipline that ensures that the state of the software at any given time is known and reconstructable. This refers both to software under development, and released.
- **Version control** (or revision control, or source control) is the management of documents or programs, keeping track of all the changes.
- **Release control** manages the different releases of product lines, having a common set of files, and different build environments.

394

394

Centralized Version Control System

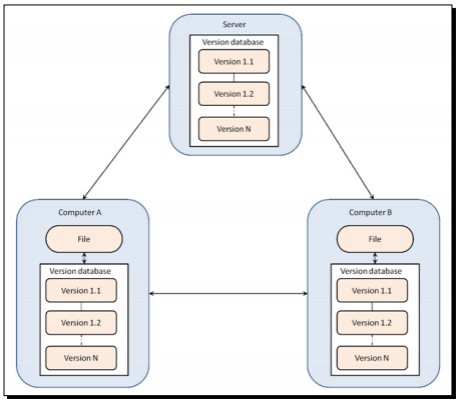


- Only the last version of the files are retrieved.
- Offers visibility on what other people are working towards.
- Any changes made to the files are automatically shared with other individuals as well.

395

395

Distributed Version Control System



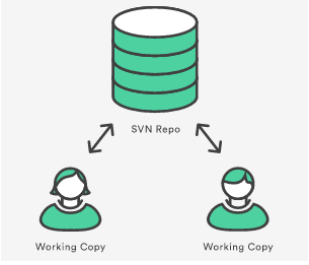
- Stores the entire history of the files on each and every machine locally
- Syncs the local changes made by the user back to the server whenever required so that the changes can be shared with others.
- Makes local changes without any concern of full time connectivity to the server
- Does not rely on a single copy of files stored in the server

396

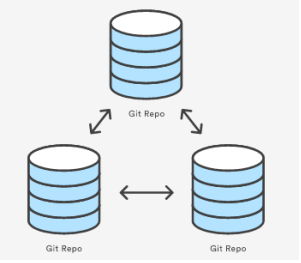
396

SVN vs Git

- Apache Subversion (SVN)
  - Open source project, founded in 2000 by CollabNet. Released in 2004
  - Centralized
  - Delta storage



- Git
  - Created by Linus Torvalds in 2005 for development of the Linux kernel, as open-source
  - Distributed
  - DAG storage



398

398

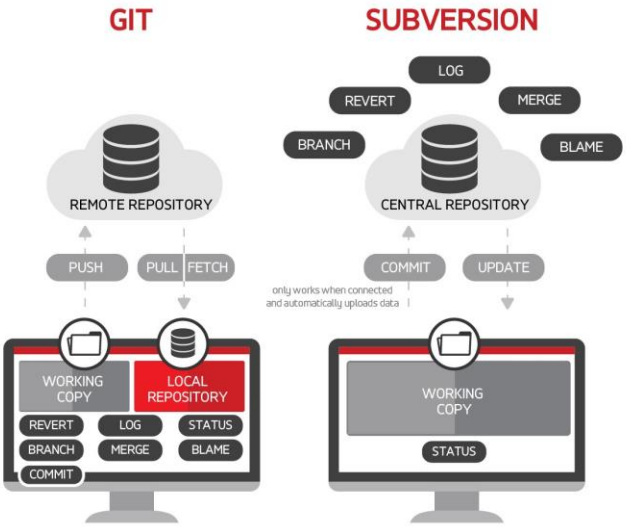
GIT vs SVN

Git vs. SVN Commands

Task	SVN Commands	Git Commands
Create a new repository.	svnadmin create	git init
Copy files into the client workspace.	svn checkout <URL> <target_name>	git clone or git fetch
Send changes to the depot.	svn commit	git commit
Add a new file.	svn import	git add
Compare changes to files.	svn diff	git diff

399

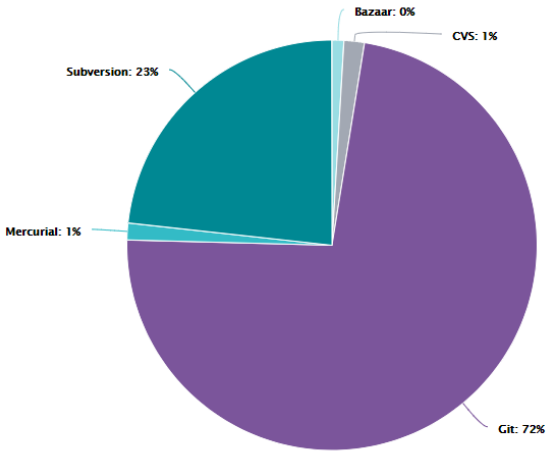
GIT vs SVN



400

GIT vs SVN

estimation of all open source repositories



401

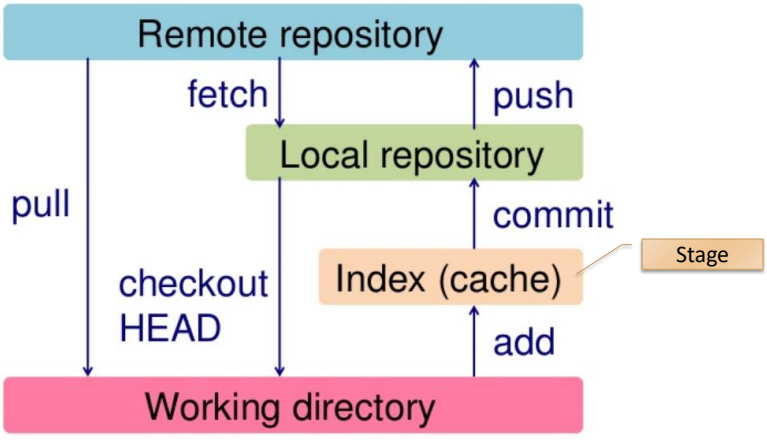
GIT vs SVN

Learning curve



402

Git



403

403

Git: Working on a repository

- Setup a repository
  - **git init:** initialize a new repository
  - **git clone:** copy a repository
  - **git config:** configure repository
- Save changes in files
  - **git add**: adds a change in the working directory to the staging area
  - **git commit**: commits the staged snapshot to the project history
- Inspect the repository
  - **git status:** displays the state of the working directory and the staging area
  - **git log:** command displays committed snapshots
- Undo changes
  - **git checkout:** fetch an old version of a file or the entire working directory from a commit

404

404

Collaborating

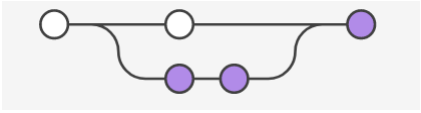
- Syncing
  - **git remote:** create, view, and delete connections to other repositories
  - **git fetch:** imports commits from a remote repository into your local repo
  - **git pull: fetch** the remote copy of the current branch **+ merge** it into the local copy
  - **git push:** transfer commits from your local repository to a remote repo.
- Pull Request
  1. A developer creates the feature in a dedicated branch in the local repo.
  2. The developer pushes the branch to a public repository.
  3. The developer files a pull request via the repository.
  4. The rest of the team reviews the code, discusses it, and alters it.
  5. The project maintainer merges the feature into the official repository and closes the pull request.

405

405

Branches

- **git branch:** create, list, rename, and delete branches
- **git checkout:** updates the files in the working directory to match the version stored in that branch
- **git merge:** take the independent lines of development created by git branch and integrate them into a single branch.

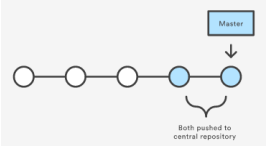


406

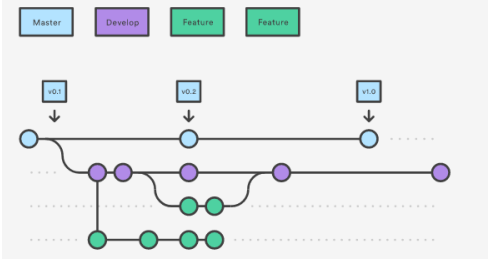
406

Workflows

- Centralized Workflow



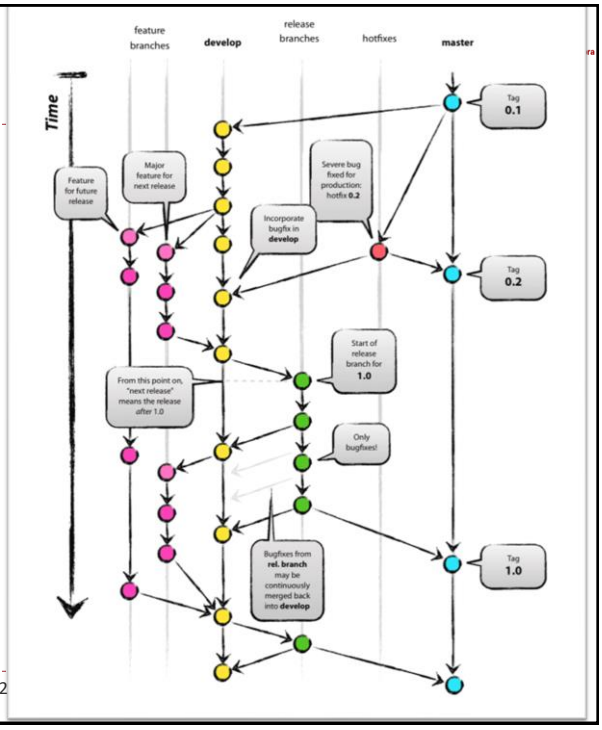
- Feature based workflow



407

407

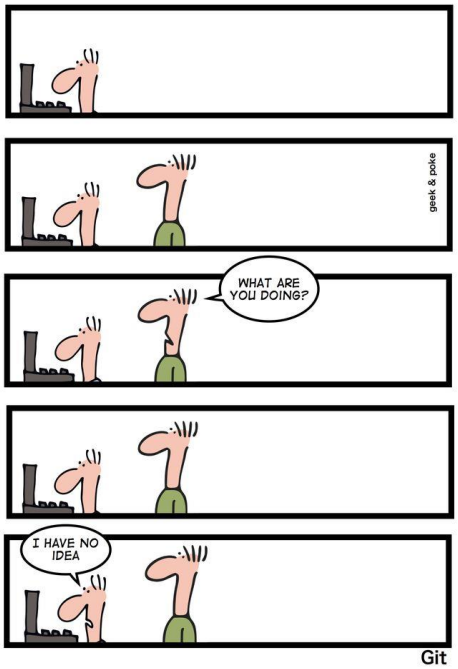
A typical Git branch model



408

SIMPLY EXPLAINED

isec Politécnico de Coimbra



Git

409

Further Reading

- GitHub
  - https://guides.github.com/activities/hello-world/
  - https://octoverse.github.com
- Git
  - http://nvie.com/posts/a-successful-git-branching-model/
  - https://www.atlassian.com/git/tutorials
- NetBeans
  - https://netbeans.org/kb/docs/ide/git.html#push
- GitKraken
  - https://www.gitkraken.com/learn-git

Some slides adapted from A. Stellman & J.Greene, "Applied Software Project Management" and Ian Sommerville, "Software Engineering"